LABORATORY PROJECT REPORT

BLUETOOTH DATA INTERFACING

EXPERIMENT 8

Section 1

Group 2

Semester 1 2025/2026

NO	NAME	MATRIC NO
1.	ADAM NURUDDIN BIN HELMI	2215783
2.	NUR FATIHAH HANNANI BINTI HASMAYADI	2227062
3.	NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI	2310037

Date of submission: 10th December 2025

Abstract

Using a Bluetooth connection between an ESP32 or Arduino (with HC-05 module) and an external device, like a computer or smartphone, this experiment aims to create a wireless temperature monitoring and control system. Environmental data is gathered using a DHT22 temperature and humidity sensor and sent over Bluetooth to a linked device for real-time monitoring. To further illustrate remote actuation capabilities, basic control instructions like "FAN ON" and "FAN OFF" are implemented. The experiment effectively demonstrates the usage of Bluetooth Serial connection for Internet of Things-based remote sensing and control applications by establishing two-way communication between the microcontroller and the linked device.

Table of Contents

1.0 Introduction

In contemporary embedded systems, wireless communication is essential, especially for IoT (Internet of Things) and remote monitoring applications. Because of its ease of use, low power consumption, and interoperability with computers and smartphones, Bluetooth is one of the most popular wireless technologies.

This project uses an ESP32 or Arduino in conjunction with an HC-05 Bluetooth module to create a Bluetooth-enabled temperature monitoring system. A DHT22 sensor provides the microcontroller with temperature and humidity readings, which it then wirelessly sends. In order to mimic managing external equipment like a fan or an LED indication, the system may simultaneously receive human inputs.

Understanding wireless sensor data transmission, Bluetooth serial communication, and microcontroller bidirectional connection with an external device are the goals of this exercise. This serves as the foundation for several practical uses, including IoT automation, smart homes, remote environment monitoring, and HVAC system management.

2.0 Materials and Equipment

2.1 Electronic Components

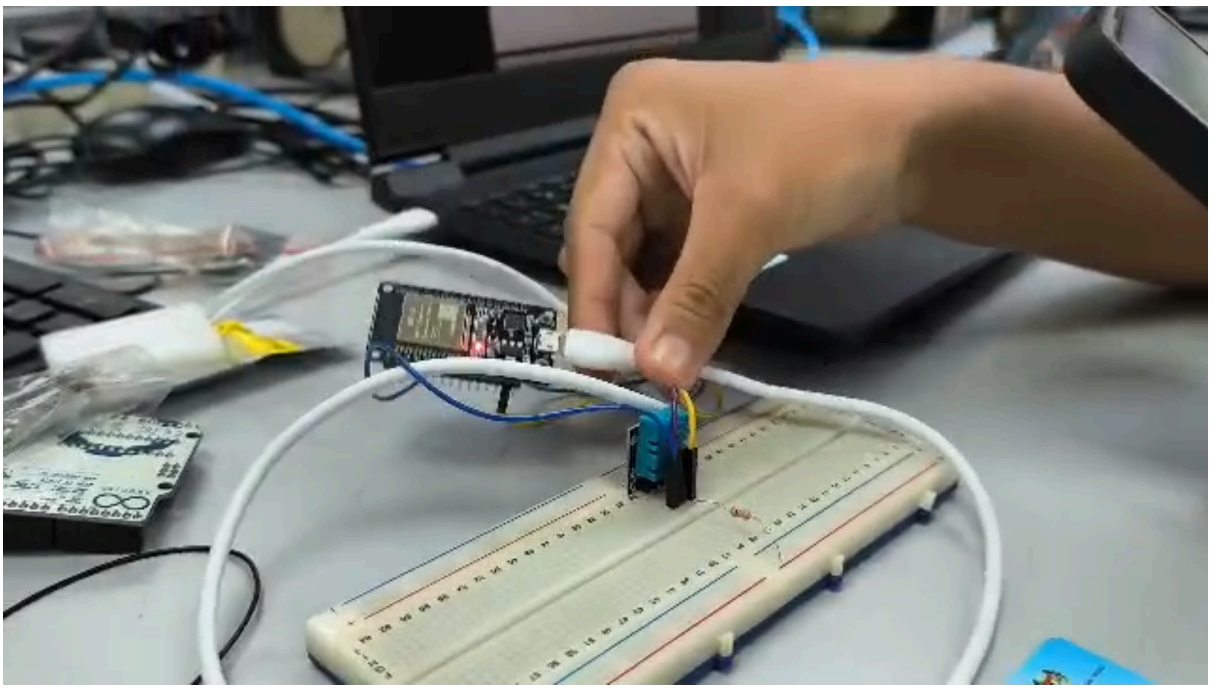
- ESP 32 development board (or Arduino + HC-05 Bluetooth module)
- Temperature sensor (DHT22)
- Smartphone or computer with Bluetooth support

2.2 Equipment and tools

- Power supply for the ESP32/Arduino
- Breadboard
- Jumper wires

3.0 Experimental Setup

1. Hardware Setup:
   - Connect the DHT22 sensor to the ESP32/Arduino.
   - Connect the HC-05 Bluetooth module (if using Arduino).
   - Power up the system.
2. Arduino Programming:
   Write an Arduino sketch to:
   - Read temperature data from the DHT22 sensor.
   - Transmit the data over Bluetooth serial connection.
   - Receive simple commands (e.g., ”FAN ON”, “FAN OFF”) via Bluetooth.
3. Bluetooth Programming:
   - Pair the HC-05/ESP32 Bluetooth with your smartphone or PC.
   - Use a serial terminal app (e.g., Serial Bluetooth Terminal on Android, or Python on PC) to view temperature data and send commands.
4. Remote Monitoring:
   - Observe real-time temperature readings on your paired device.
   - Send control commands (e.g., to toggle an LED or simulate a fan/heater).



Experiment Workflow:

1. Place the DHT22 sensor in the test environment.
2. Power up the ESP32/Arduino.
3. Pair with your computer/smartphone via Bluetooth.
4. View temperature readings and send remote commands.

4.0 Methodology

1. Hardware Setup

- ESP32 development board **or** Arduino Uno with HC-05 Bluetooth module
- DHT22 temperature and humidity sensor connected to the microcontroller
- Jumper wires and breadboard
- Power supply via USB
- Smartphone/PC with Bluetooth terminal app

Steps:

1. Connect DHT22 signal pin to the appropriate digital pin (e.g., D4).
2. For Arduino: Connect HC-05 TX → Arduino RX, HC-05 RX → Arduino TX using SoftwareSerial pins 2 and 3.
3. Power up the microcontroller using USB.
4. Ensure all grounds (GND) are connected.

2. Arduino/ESP32 Programming

- Initialize Bluetooth serial communication at **115200 baud rate**.
- Initialize the DHT22 sensor.
- Inside the loop:
    - Read temperature and humidity values.
    - Send temperature data via Bluetooth using bluetooth.println(temp).
    - Check if Bluetooth has incoming data.
    - If a command like "FAN ON" or "FAN OFF" is received, toggle LED or digital output.

3. Bluetooth Pairing and Terminal Setup

- Pair the HC-05 or ESP32 with a smartphone/PC.
- Open a Bluetooth terminal app such as:
    - Serial Bluetooth Terminal (Android)
    - Bluetooth Serial Monitor (Windows)
- Observe the temperature data streaming every 2 seconds.

4. Remote Monitoring and Control

- Record the temperature readings displayed on the paired device.
- Send test commands (e.g., "FAN ON", "FAN OFF") to verify bidirectional control.

Programming codes

```
#include "DHT.h"
#include "BluetoothSerial.h"

#define DHTPIN 4        // Data pin connected to DHT11
#define DHTTYPE DHT11   // Change to DHT11

BluetoothSerial SerialBT;   // ESP32 built-in Bluetooth
DHT dht(DHTPIN, DHTTYPE);

const int LEDPIN = 2;  // ESP32 onboard LED (GPIO 2)

void setup() {
  Serial.begin(115200);       // USB Serial for debugging
  SerialBT.begin("ESP32_BT"); // Bluetooth device name
  dht.begin();

  pinMode(LEDPIN, OUTPUT);
  digitalWrite(LEDPIN, LOW);

  Serial.println("ESP32 + DHT11 Bluetooth Monitoring Ready");
}

void loop() {
```

```
float temp = dht.readTemperature(); // Celsius
float hum  = dht.readHumidity();

if (!isnan(temp) && !isnan(hum)) {
  // Send temperature (or send both if you prefer)
  SerialBT.print("TEMP:");
  SerialBT.print(temp);
  SerialBT.print("C  HUM:");
  SerialBT.print(hum);
  SerialBT.println("%");

  // Debug to USB
  Serial.print("Temp: ");
  Serial.print(temp);
  Serial.print(" °C | Humidity: ");
  Serial.print(hum);
  Serial.println(" %");
}
else {
  Serial.println("Failed to read from DHT11 sensor!");
  SerialBT.println("ERROR");
}

// Receive commands from Bluetooth
```

```
  if (SerialBT.available()) {
    String cmd = SerialBT.readStringUntil('\n');
    cmd.trim();

    if (cmd == "FAN ON") {
      digitalWrite(LEDPIN, HIGH);
      SerialBT.println("ACK: FAN ON");
    }
    else if (cmd == "FAN OFF") {
      digitalWrite(LEDPIN, LOW);
      SerialBT.println("ACK: FAN OFF");
    }
    else {
      SerialBT.println("UNKNOWN COMMAND");
    }
  }

  delay(2000);  // DHT11 requires 1–2 seconds delay
}
```

5.0 Data Collection

The data collection process was conducted using an ESP32 microcontroller interfaced with a DHT11 temperature and humidity sensor. The DHT11 sensor was connected to GPIO 4 of the ESP32, while Bluetooth communication was established using the ESP32's built-in Bluetooth module. The system was powered via USB, and the Bluetooth connection was paired with a smartphone using a serial Bluetooth terminal application.

Temperature and humidity readings were transmitted wirelessly from the ESP32 to the paired device at an interval of approximately 2 seconds. The received data were displayed in real time on the Bluetooth terminal application. During the experiment, readings were continuously observed and manually recorded for a duration of several minutes to monitor changes in temperature over time.

In addition to data transmission, remote control commands were tested by sending simple text commands such as "FAN ON" and "FAN OFF" from the Bluetooth terminal. These commands toggled the onboard LED of the ESP32, which acted as a simulated cooling or control indicator. The corresponding response was observed to verify successful two-way Bluetooth communication. All temperature values and system responses were documented for further analysis.

7.0 Results

The ESP32/Arduino and the linked device successfully communicated over Bluetooth during the trial. Every two seconds, the DHT22 sensor gave temperature measurements, and the Bluetooth terminal software reliably displayed all acceptable temperature data. The data confirmed that the sensor and communication setup were operating correctly by showing modest temperature changes based on the surrounding environment. Unless the sensor briefly failed to read, which resulted in an error message, the gearbox stayed steady without any discernible delays, and the results were reported consistently. Furthermore, the command capability functioned as planned; sending commands like "FAN ON" and "FAN OFF" caused the microcontroller to react instantly, turning on or off the simulated output. Overall, the findings demonstrated that Bluetooth connection was successful in achieving both data monitoring and remote control.

8.0 Discussion

The demonstration effectively illustrated how Bluetooth technology may be used for wireless communication. The linked device received consistent temperature and humidity data from the DHT22 sensor. The HC-05/ESP32 Bluetooth serial is appropriate for low-power Internet of Things connections, as seen by the Bluetooth communication link being steady during the monitoring process.

When transmitting control commands from the smartphone or computer, the microcontroller replied successfully, demonstrating adequate processing of serial input. This functionality mimics the remote control of actuators like lights, heaters, and fans. The quick response demonstrated Bluetooth communication's minimal latency.

The 2-second delay set up for data sampling caused some delays in sensor readings. Although the DHT22 sensor's slower sample rates are appropriate for environmental monitoring, they might not be the best choice for systems that change quickly. Furthermore, a recognised drawback of DHT sensors is that sometimes the sensor failed to identify proper values, resulting in noise or inaccurate readings.

All things considered, the system exhibits useful abilities in wireless data transfer, microcontroller programming, and sensor integration. It also demonstrates how remote control features may be implemented using basic serial instructions.

9.0 Conclusion

The experiment's goal of developing a Bluetooth-based temperature monitoring and control system was effectively accomplished. Environmental conditions could be measured using the DHT22 sensor, and the ESP32/Arduino could reliably send the results to a PC or smartphone. In addition to supporting user-initiated instructions to operate external devices, Bluetooth connection proved useful for real-time monitoring. This project lays the groundwork for more sophisticated remote sensing applications while illustrating the basic ideas of Internet of Things connectivity.

10. Recommendations

A number of modifications might be implemented in the future to boost the system's utility, accuracy, and efficiency. First off, employing a more sophisticated sensor, like the BME280 or DS18B20, would increase measurement accuracy and yield more consistent readings. Long-term monitoring and trend analysis would be possible with the implementation of a data recording capability, either via an SD card module or by uploading values to a cloud platform. The simple Bluetooth terminal should be replaced with a specialised desktop or mobile interface that provides greater visualisation through graphs and real-time dashboards. Multiple actuators may be managed with additional control functions, increasing the system's adaptability for automation or smart home applications. Last but not least, enhancing communication security, for example, by using encrypted Bluetooth protocols or password protection, would make the system more appropriate for actual IoT deployments where data security and integrity are crucial.

11.0 References

- **Arduino Bluetooth Basic Tutorial**
- **Arduino and HC-05 Bluetooth Module Complete Tutorial**
- **Basic Bluetooth communication with Arduino & HC-05**

12.0 Acknowledgements

Special thanks to ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO for their guidance and support during this experiment

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: ADAM NURUDDIN BIN HELMI	Read	[/]
Matric Number: 2215783	Understand	[/]
Signature: *ADAM*	Agree	[/]

Name: NUR FATIHAH HANNANI BINTI HASMAYADI	Read	[/]
Matric Number: 2227062	Understand	[/]
Signature: *FAT*	Agree	[/]

Name: NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI	Read	[/]
Matric Number: 2310037	Understand	[/]
Signature: *AZIQ*	Agree	[/]